

INTERJÚ-FELKÉSZÍTŐ · MOBIL KIADÁS

Technikai interjú- felkészítő

Full Stack Developer — Redspher

TypeScript · React · Node.js · AWS · REST · AI / LLM /
MCP / agentek · CI/CD · PHP legacy

Készült Richardnak · 2026. július

Contern, Luxemburg · Logisztika / on-demand fuvarozás

Telefonos olvasásra optimalizálva — keskeny oldalak,
nagy betűméret.

A hangosan gyakorlandó szövegpanelek angol eredetije
az angol verzióban.

Tartalom

1 · A pozíció, dekódolva	3
2 · Pitch és bizonyítéktérkép	5
3 · TypeScript	8
4 · React	14
5 · Node.js	20
6 · REST API-tervezés	26
7 · AWS	30
8 · AI: LLM-ek, MCP, agentek	36
9 · Git, CI/CD, Docker	46
10 · A PHP legacy szál	48
11 · Rendszertervezés, logisztikai ízzel	51
12 · Viselkedéses és üzleti kérdések	54
13 · Kérdések feléjük	57
14 · A cég dióhéjban	58
15 · Puska és záró checklist	59

Használat: fúsd át a Q&A kártyákat, mondd ki hangosan a válaszokat, csillagozd, ami bizonytalan. Zöld doboz = a személyes szövegpaneljeid. Sárga doboz = őszinteségi korlátok. Kék doboz = kitöltendő sztorivázak. A szó szerint bema-golandó idézeteket élesben angolul mondd — az eredetijük az angol PDF-ben van.

1 · **A pozíció, dekódolva**

Mit mond valójában az álláshirdetés.

A Redspher **termékszemléletű full-stack fejlesztőt** keres, nem tiszta specialistát. A hirdetés sorai között olvasva:

- **„Business-oriented development team” + „pragmatic mindset”** — azt fogják vizsgálni, hogyan fordítasz homályos üzleti igényt szállítható szoftverre, és hogyan hozol trade-off döntéseket. Számíts rá, hogy a viselkedéses kérdéseknek komoly súlyuk lesz.
- **„Strong TypeScript and React”** — ez a kémény lécz. Itt várható a legmélyebb technikai fúrás, valószínűleg élő kódolással vagy code review feladattal.
- **„Backend services” + Node.js + REST + AWS** — szolid medior-szenior backend-tudást várnak, de a hirdetés a frontenddel nyit, így a backend-kérdések inkább gyakorlatiasak lesznek (API-tervezés, üzenetsorok, adat), nem mély elosztottrendszer-elmélet. Itt a lécz fölé tudsz menni — ez az előnyöd.
- **AI: „MCP, AI agents, orchestration”, „explore and prototype”** — szokatlan, hogy egy álláshirdetés név szerint említi az MCP-t. Olyan embert keresnek, aki felelősen tud AI-funkciókat prototipizálni. Nagyon kevés jelöltnek van valódi MCP-tapasztalata; **te saját MCP szerveret üzemeltetsz**. Ezzel nyiss.

- **„PHP ... legacy systems, is a plus”** — van egy örökölt PHP-állományuk és egy folyamatban lévő migrációs történetük. A korai éveid PHP-ja plusz egy tiszta strangler-fig narratíva a „pluszból” megkülönböztetőt csinál.
- **„International environment”** — luxemburgi cég, sok nemzetiség; a Docler Luxembourgnál töltött éveid erre közvetlen válasz.

Az interjúfolyamat várható alakja

Ennél a profilnál tipikus: (1) HR/kultúra-szűrő, (2) technikai interjú TS/React/Node/AWS témákban + AI-beszélgetés, (3) élő kódolás vagy take-home (React komponens vagy kis TS API), (4) csapat/vezetői kör az üzleti együttműködésről. Készülj mind a négyre; ez a doksi a 2-4-et fedi le.

2 · Pitch és bizonyítéktérkép

A 60 másodperces bemutatkozásod, és bizonyíték a hirdetés minden sorára.

60 másodperces bemutatkozás (élesben angolul mondod — az eredeti az angol PDF §2-ben; itt a tartalma)

„Senior full-stack mérnök vagyok 16 év tapasztalattal, legutóbb a luxemburgi Docler Holdingnál, ahol Senior Frontend és Senior Backend Engineer szerepet is betöltöttem. Frontenden professzionálisan Reacttel dolgoztam; backenden komoly méretű Node.js szolgáltatásokat vittem — köztük egy valós idejű eseményelosztó rendszert nagyjából 50 000 párhuzamos klienssel, és egy ügyfélszolgálati chat teljes, végponttól végpontig tartó újraépítését, ami az infrastruktúra-költséget kb. 75%-kal csökkentette. Dupla AWS-tanúsítványom van — Solutions Architect és Developer Associate — éles Lambda- és S3-munkával. Az utóbbi időszakban pedig mélyen beleástam magam az AI-támogatott fejlesztésbe: napi szinten használok AI agenteket, belső előadásokat tartottam AI-ról és neurális hálókról, és megépítettem és üzemeltetem a saját MCP szerveremet, ami egy dokumentumgeneráló pipeline-t tesz elérhetővé toolokként LLM-kliensek számára. Ez a kombináció — TypeScript a teljes stacken, AWS, és kézzelfogható MCP/agent-munka — pontosan az, amiért ez a pozíció megfogott.”

Követelmény → a bizonyítékok

Amit kérnek **Amit hozol**

Full-stack tapasztalat	16 év; külön Senior Frontend és Senior Backend szerep a Doclernél
Erős TypeScript/React	Professzionális React-évek a Doclernél (nem önképzés); TS a jelenlegi projektjeiben végig
Node.js backend	Nagy forgalmú broadcaster (~50k párhuzamos kliens, ~100k üzenet/mp kimenő); chat-újraépítés (Redis pub/sub WebSocket, –75% infra-költség, –30% p95)
AWS	SAA + DVA tanúsítvány; éles Lambda + S3 a Doclernél; a serverless stack többi része tanúsítási labokból
REST API-k	Évek API-munkája; Distributed Checkout referencia-projekt (NestJS, outbox, saga, OIDC)
AI / LLM / MCP / agentek	Saját MCP szerver éles használatban (prompt-cv pipeline); napi agentikus AI-munkafolyamat; belső AI-előadások; spec-vezérelt agent-fejlesztés
Git, CI/CD	GitHub Actions + Helm + Argo CD GitOps a személyes platformodon; professzionális Git a teljes karrieren át
PHP legacy (plusz)	PHP a WebGarden-időszakból; kényelmes legacy-karbantartás és -integrálás TS-re migrálás közben
Üzletvezérelt, nemzetközi	Idézhető költség/latencia-eredmények; évek Luxemburg multikulturális tech-szcénájában

Összintéségi korlátok — ezeket ne lépd át

- **Két külön sztori, soha nem keverve:** a chat-újraépítés (Redis pub/sub, alacsony forgalom,

–75% költség, –30% p95, teljes ownership) és az eseménybroadcaster (~50k párhuzamos kliens, ~100k üzenet/mp, részleges újratervezés, ~10% gyorsabb kézbesítés). A számok keverése a visszakérdezéseknél bomlik ki.

- **AWS:** a Lambda + S3 valódi éles munka. Az API Gateway, DynamoDB, SQS/SNS, EventBridge, Step Functions, CloudWatch *tanúsítvány* + *gyakorlati lab* — pontosan ezt mondd, magabiztosan: „tanúsítva vagyok belőle, labokban gyakoroltam, élesben még nem futtattam.”
- **Soha ne állíts** éles Terraform-, OpenTelemetry-, DDD-, C4- vagy ADR-tapasztalatot. Ha kérdezik: „ismerem a koncepciókat, élesben nem gyakoroltam őket.”
- **SQL/Oracle** = egyetem + folyamatos önképzés, nem éles DBA-szint. Keretezés: „stabil, munkára fogható SQL, aktívan élesítem.”
- A **Kubernetes/IaC** tudás forrása: DVA-tanúsítvány, New Horizons képzés és a k3s/Argo CD személyes platformod — önmagában is remek, őszinte sztori.

Miért nyer itt az őszinteség

A „pragmatikus” csapatok azonnal kiszagolják a fel-fújást. Egy határozott „élesben még nem futtattam, de így állnék neki”, jól megválaszolva, *többet* ér a blöffnél — pontosan azt az önismerő, üzletileg biztonságos ítélőképességet mutatja, amiért felvesznek.

3 · TypeScript

A kemény léccel. Cél: feszes, szenior szintű válaszok, mindegyikhez egy konkrét példa.

K. Miért TypeScript egy üzleti terméken, a „kevesebb bug”-on túl?

A típusok futtatható dokumentáció és refaktorálási védőháló: egy mező átnevezése vagy egy API-kontraktus módosítása fordító által vezetett feladat lesz grep-és-imádkozás helyett. Üzleti szabályokat kódolnak (az `OrderStatus` union, nem `string`), így az érvénytelen állapot le sem fordul. Kulcsfontosságú, hogy a típusok **futás-időben törölődnek** — ezért minden határon (HTTP, queue, env változók) továbbra is validálsz valami Zod-szerűvel, és a statikus típust a sémából származtatod: egyetlen igazságforrás.

K. `interface` vagy `type`?

Objektum-alakokra nagyrészt felcserélhetők. Az `interface` támogatja a declaration merginget, és idiomatikus publikus/bővíthető kontraktusokhoz; a `type` kell unionokhoz, metszetekhez, mapped és conditional típusokhoz, tuple-ökhöz. Az ökölszabályom: `interface` az objektum-kontraktusokra, `type` alias minden másra — és mindenekelőtt egyetlen konzisztens csapatkonvenció.

K. any VS unknown VS never ?

Az `any` mindkét irányban kikapcsolja a fordítót — megfertőz mindent, amihez hozzáér. Az `unknown` a biztonságos top type: bármit ráadhatsz, de használat előtt szűkítened kell — külső inputhoz ez a helyes típus. A `never` az üres típus: elérhetetlen kód, kimerítő ellenőrzések maradéka. Szigorú kódbázisban `any` csak tudatosan karanténózott határokon jelenhet meg.

K. Mutass generikust constrainttel.

```
function pluck<T, K extends keyof T>(
  items: T[], key: K
): T[K][] {
  return items.map(i => i[key]);
}
// pluck(orders, "status") -> OrderStatus[]
```

A `K extends keyof T` a kulcsot az objektum valódi property-jeihez köti, így az elgépelés fordítási hiba, a visszatérési típus pedig pontosan következtetett.

K. Mely utility type-okat használsz ténylegesen?

`Partial` / `Required` patch payloadokhoz, `Pick` / `Omit` DTO-k származtatásához a domain-modellből, `Record<K,V>` kulcsolt map-ekhez, `ReturnType` / `Parameters` függvények wrappelésakor, `Awaited` promise-ok kibontásához, `Readonly` immutábilis nézetekhez. Az elv: egyetlen kanonikus alakból származtass, ahelyett hogy kézzel másolt közeli duplikátumok széttartanak.

K. Diszkriminált unionok — miért szeretik a szeniorok?

```
type ShipmentEvent =  
  | { type: "picked_up"; at: string }  
  | { type: "delayed"; at: string; reason:  
    string }  
  | { type: "delivered"; at: string; pod:  
    string };  
  
switch (e.type) {  
  case "picked_up": ...  
  case "delayed": ...  
  case "delivered": ...  
  default: { const x: never = e; return x; }  
}
```

A `type` tag minden ágban automatikusan szűkít, a `never` default pedig **kimerítővé** teszi a switchet: új eseményvariánst felvéve minden lekezeletlen switch fordítási hibává válik. Tökéletes illeszkedés logisztikai eseményfolyamokhoz.

K. Type guardok és szűkítés (narrowing)?

Beépített szűkítés `typeof`, `instanceof`, `in` és diszkrimináns-ellenőrzés útján. Az egyéni guard `value is X`-et ad vissza; az `assertion function` (`asserts value is X`) `false` helyett dob — invariánsokhoz jó. Csapda: egy rossz kézi guard hazudik a fordítónak, ezért külső adatra séma-validálást preferálok (Zod `safeParse`), ami szűkít és ténylegesen ellenőriz is.

K. `satisfies` VS `as`?

Az `as` `assertion` — felülbírálja a fordítót, és valódi hibákat rejthet el. A `satisfies` ellenőrzi az értéket egy típus ellen, *miközben megőrzi a szűkebb következtetett típust*: a `const config = {...}` `satisfies AppConfig` validálja az alakot, de megtartja a literál típusokat az autocomplethez. `satisfies` `config`-objektumokra és lookup-táblákra; az `as` maradjon arra a ritka határra, ahol tényleg többet tudsz a fordítónál (plusz `as const` a literál-fagyasztáshoz).

K. Mely strict flagek számítanak neked?

A `strict: true` az alap — mindenekelőtt a `strictNullChecks`, ami a milliárd dolláros hibát fordítási hibává teszi. Ezen túl a `noUncheckedIndexedAccess`-t értékelem (tömb/record-hozzáférés `T | undefined`-ot ad, kikényszerítve a hiányzó eset kezelését) és az `exactOptionalPropertyTypes`-t. E kettő említése jelzi, hogy éles körülmények közt futtatál strict TS-t, nem csak bekapcsoltad a defaultokat.

K. A TypeScript strukturálisan típusos — mik a következmények?

A kompatibilitás alak szerinti, nem név szerinti: bármely objektum megfelel, amelynek jók a tagjai. Rugalmasságnak remek; kockázatos, ha két alak véletlenül egybeesik (`CustomerId` vs `OrderId`, mindkettő `string`). Ellenszerek: `excess-property check` objektum-literálok, és **branded type-ok** (`type OrderId = string & { __brand: "OrderId" }`) ott, ahol az ID-k keverése valódi üzleti bug lenne.

K. Hogyan migrálnál egy JS (vagy legacy-közei) kódbázist TS-re?

Inkrementálisan: `allowJs` + `checkJs` bekapcsolása, fájlankénti konverzió a legtöbbet megosztott moduloktól indulva, `.d.ts` a típusozatlan belsők-höz, a szigorúság fokozatos felhúzása (lazán indulva, flagenként húzva), és CI-kapu: „nincs új any”. Soha nem big-bang újraírás — a kódbázisnak végig szállíthatónak kell maradnia. Ugyanez a racsniszemlélet érvényes az ő legacy PHP-juk típusos API-kontraktusok mögé csomagolására is.

K. Hogyan tartod szinkronban az API-típusokat frontend és backend között?

Egy igazságforrás, kifelé generálva: vagy közös csomagból megosztott Zod-sémák (ebből következik a runtime-validáció és a statikus típus is), vagy OpenAPI-spec generált kliensekkel/típusokkal. Amit kerülök: kézzel karbantartott, duplikált interface-ek a két oldalon — mindig széttartanak, és a fordító nem lát át a HTTP-résen.

4 · React

Itt várható a legmélyebb fűrés. A válaszaidat a renderelési modellhez horgonyozd.

K. Vezess végig a React renderelési modelljén.

State vagy props változik → a React újrendereli a komponenst és alából az egész al-fáját → új elemfa készül → a rekonsziliáció diffeli az előzővel → csak a valódi különbségek kerülnek a DOM-ba. A key-k stabil identitást adnak a listaelemeknek renderek között, így a state és a DOM-node újrahasznosul, nem semmisül meg. Két következmény: a render-függvényeknek tisztának kell lenniük (többször is lefuthatnak), és a teljesítmény-munka vagy a state lejjebb mozgása, vagy határok memoizálása — nem a DOM-mal harcolás.

K. useEffect — hogyan használd helyesen?

Az effect a komponenst egy *külső* rendszerrel szinkronizálja: feliratkozások, timerek, imperatív API-k. Szabályaim: kimerítő dependency-lista (bekapcsolt lint-szabállyal), cleanup mindenhez, ami feliratkozik, és a StrictMode dev-beli dupla mount-unmountját ingyen tesztnek tekintem arra, hogy a cleanup működik. Amire az effect **nem** való: származtatott state (számold render közben), user-eseményre reagálás (a handlerben csináld), és state-frissítések láncolása kaszkádolt effecteken át — az a klasszikus spagettiforrás.

K. Mikor segít ténylegesen a `useMemo` / `useCallback` / `React.memo`?

A referenciális egyenlőségért léteznek: a `React.memo` kihagyja az újrenderelést, ha a propok shallow-egyenlők, ezért a memoizált gyerekeknek átadott callbackek/objektumok `useCallback` / `useMemo`-t igényelnek a stabil identitáshoz. Ezenkívül tényleg drága számításokra és effect-dependencyk stabilizálására. De a memoizálás nem ingyenes — extra memória és összehasonlítás, az idő előtti szórás pedig valódi strukturális bajokat rejt el. Sorrendem: mérés Profilerrel, először a state-elhelyezés átstrukturálása, végül a bizonyítottan forró határok memoizálása.

K. Hogyan kezeled a state-et egy komoly React-appban?

Először válaszd szét a **szerver-state-et** és a **kliens-state-et**. A szerver-state (rendelések, ajánlatok, szállítmányok) data-fetching cache-be való, mint a TanStack Query: cache-elés, deduplikáció, háttér-refetch, invalidálás, optimista frissítés — megoldott problémák, ne építsd újra Reduxban. A kliens/UI-state maradjon lokális, ameddig lehet; ha osztani kell, valami könnyű, mint a Zustand, vagy Redux Toolkit ott, ahol a csapat már arra standardizált. A Context alacsony frekvenciájú globálokra való (téma, auth-identitás), nem state-manager.

K. Mi a Context csapdája?

Minden consumer újrenderel, valahányszor a context értéke változik — és egy inline `value={{...}}` objektum minden rendernél változik. Ellenszerek: memoizált érték, egy kövér context szétvágása több keskenyre (state vs dispatch), vagy feliratkozás-alapú store selectorokkal, ha a frissítés gyakori. A Context terjeszt; nem optimalizál.

K. Controlled vs uncontrolled inputok — és az űrlapok általában?

Controlled: a React state az igazságforrás (`value` + `onChange`) — teljes kontroll, de minden billentyűleütés renderel. Uncontrolled: a DOM tartja az értéket, ref-fel olvasod. Valódi üzleti űrlapokhoz react-hook-formot használok: a motorháztető alatt uncontrolled, ezért gyors, séma-validálással (Zod resolver), így az űrlapszabályok és az API-kontraktusok egy definíción osztoznak.

K. Code splitting és hibakezelés a UI-ban?

Route-szintű splitting `React.lazy` + `Suspense` -szel tartja becsületesnek a kezdeti bundle-t; a nehéz widgetek (chartok, térképek) külön splitelve. Az error boundary route-okat/feature-héjakat csomagol, így egy összeomló widget fallbackre degradál, nem fehér képernyőt okoz; párosítsd reportinggal, hogy a hibák láthatók legyenek, ne elnyelve.

K. Mit változtatott a React 18/19, amit ténylegesen használsz?

React 18: automatikus batching mindenhol, `useTransition` / `useDeferredValue`, hogy a gépelés rezszponzív maradjon drága listák újrenderelése közben, `useId`, és az érő `Suspense`. React 19: a form Actionök `useActionState` / `useOptimistic` -kal csökkentik a kézzel írt pending/error state-et, `use()` promise-ok olvasására, a ref normál prop, és a Server Componentek mainstreammé válása Next.js-féle frameworkökön át — az RSC-t így keretezd: „ismerem és használtam a saját Next.js App Router projektben, pragmatikusan vezetném be ott, ahol a csapat frameworkje támogatja”.

K. Lassú egy oldal. Mi a debug-munkamented?

Reprodukálás → React DevTools Profiler: mi renderel és miért (mely propok/state változtak) → tipikus tettesek: túl magasan élő state, instabil objektum/callback-propok, amik kiütik a memót, hatalmas listák virtualizálás nélkül, layout-thrash effectekből. Strukturális javítás (state lejjebb, komponensbontás, virtualizálás `react-window`-val), a bizonyított forró útvonalak memoizálása, újramérés. Betöltési teljesítményre: bundle-analízis, code splitting és cache-headerek a mikrooptimalizálás előtt.

K. Gyakori React anti-patternek, amiket review-ban jeleznél?

- Tömbindex key-ként átrendezhető/szűrhető listán — a state átvérzik az elemek közt.
- Prop másolása state-be („tükör-state”) — azonnali elavulás; származtasd inkább.
- Effect-láncok, ahol a state-frissítés újabb effecteket triggerel — strukturáld át az adatfolyamot.
- Egyetlen 800 soros komponens fetch + űrlap + táblázat felelősséggel — bontsd felelősség szerint.
- JSX-be temetett üzleti logika — emeld ki hook-ba/tiszta függvénybe, hogy tesztelhető legyen.

K. Hogyan tesztelsz Reactet?

React Testing Library a felhasználó szemszögéből: lekérdezés role/label szerint, interakció, látható kimenetek assertálása — nem implementációs részletek, így a refaktor nem tépi szét a suite-ot. A hálózat a határon mockolva MSW-vel, hogy a komponensek a valódi fetch-útvonalakat futtassák. A kiemelt hookok és a tiszta logika közvetlen unit-tesztje; vékony E2E-réteg (Playwright) a pénzes útvonalakra, mint a checkout vagy az ajánlatbeadás.

K. TypeScript Reactben — mintáid, amikre támaszkodsz?

Típusos propok diszkriminált unionnal variáns-komponensekhez (`{ kind: "link"; href }` vs `{ kind: "button"; onClick }` — az érvénytelen kombináció le sem fordul), generikus komponensek táblázatokhoz/selectekhez (`<Table<Order>>`), `ComponentProps<typeof X>` meglévő komponensek bővítéséhez, és a szerver-state típusok következtetése a közös API-sémából, nem újradeklarálva.

5 · Node.js

Itt a lécs fölé mehetsz. Előbb vágd rá hibátlanul az event loopot, aztán jöhetnek a skálázási sztorik.

K. Magyarázd el az event loopot. (A klasszikus — ezt üsd ki.)

A Node egyetlen szálon futtatja a JS-t; a libuv hajt egy event loopot fázisokkal — **timers** (setTimeout/setInterval), pending callbackek, **poll** (I/O), **check** (setImmediate), close callbackek. A callbackek között a **microtaskok** teljesen lefutnak: először a `process.nextTick`, aztán a feloldott promise-callbackek. Maga az I/O az OS-hez vagy a libuv thread pooljához delegált (fs, DNS, crypto), tehát az „egyszálú” csak a te JS-edre igaz. Következmény: bármilyen CPU-nehéz szinkron munka *minden* kérést befagyaszt — ezért szervezed ki.

K. Akkor hogyan skálázod a Node-ot, és mit kezdesz a CPU-igényes munkával?

Processzen belül: nem blokkoló handlererek; a CPU-munka (PDF-generálás, crypto, óriásfájl-parszolás) `worker_threads`-be vagy külön szolgáltatásba/queue-ba megy. Gépen belül: magonként egy processz cluster/PM2-vel — bár konténeres világban az idióma konténerenként egy processz és horizontális skálázás load balancer mögött, health checkekkel. Sticky session csak ha muszáj (jobb: a state kiszervezése Redisbe).

K. Streamek és backpressure — mikor számítanak?

Amikor az adat nagyobb a memóriánál vagy időben érkezik: 2 milliós CSV-export, fájlfeltöltés proxyzása S3-ba, logok transzformálása. A stream korlátos memóriával dolgoz fel chunkokat; a **backpressure** a flow control — a lassú fogyasztó a pipe / pipeline szemantikán át megállítja a termelőt, ahelyett hogy korlátlanul bufferelne. Én `stream.pipeline`-t használok (vagy web streameket), mert helyesen terjeszti a hibákat és a cleanupot, amit a kézi `.pipe()` láncok híresen nem.

K. Hibakezelési stratégia egy Node-szolgáltatásban?

Async/await try/catch-csel értelmes határokon; központi error middleware/filter, ami az ismert hibaosztályokat HTTP problem-válaszokra képezi, minden mást pedig logolt 500-ra correlation ID-vel. Válaszd szét az operational hibákat (retryelhető, várható) a programozói hibáktól (crash-t érdemel). `unhandledRejection` / `uncaughtException` regisztrálva: logol és kilép — a félig törött processz rosszabb az újraindítotttnál. És **graceful shutdown**: SIGTERM-re ne fogadj új kapcsolatot, engeddd kifutni a folyamatban lévő kéréseket, zárd a DB-poolokat, majd lépj ki — zero-downtime deployhoz elengedhetetlen.

K. Express vs Fastify vs NestJS — a te olvasatod?

Express: minimális, óriási ökoszisztéma, de minden fegyelmen múltó konvenció. Fastify: gyorsabb, beépített séma-alapú validálás és szeriálizálás — karcsú szolgáltatásokhoz a defaultom. NestJS: véleményes struktúra (modulok, DI, guardok, interceptorok), ami akkor térül meg, amikor a csapat és a kód bázis nő — a Distributed Checkout referencia-projektemhez pont ezért ezt választottam, hogy éles struktúrát demonstráljak: típusos config-validálás, TypeORM csak migrációkkal, OIDC-guardok, tranzakciós outbox. Az őszinte keretezés: a framework kevésbé számít, mint a határok, amiket kikényszerítesz vele.

K. Konfiguráció és secretok?

Twelve-factor: config a környezetből, sosem commitolva; egy séma (Zod) bootkor validálja az összes env-változót, így a hiányzó változó induláskor bukik gyorsan, nem hajnali 3-kor egy requestben. A secretok a platform secret store-jából jönnek (AWS Secrets Manager / SSM vagy K8s-secretok), futásidőben injektálva; lokálisan gitignore-olt `.env`. Környezetenként külön config, ugyanaz az artifact átléptetve rajtuk.

K. A legfontosabb API-biztonsági kérdések, amikre ténylegesen lépsz?

- Minden input validálása a peremen (séma-validálás) — az injection és a type-confusion itt hal meg.
- AuthN $\neq$ AuthZ: ellenőrizd a JWT-t, *aztán* azt is, hogy ez a user hozzányúlhat-e ehhez az erőforráshoz (az IDOR az #1 valós API-lyuk).
- Rate limiting és payload-méretkorlát; helmet-jellegű security headerek.
- Paraméterezett query-k / ORM — soha stringből épített SQL.
- Függőség-higiénia: lockfile-ok, `npm audit` / Dependabot a CI-ban, minimális base image-ek.
- Se secret, se PII a logokban — GDPR-terep, főleg ügyfél-szállítmányadatokkal.

K. Hogy néz ki az observability a szolgáltatásaidban?

Strukturált JSON-logok (pino) request/correlation ID-vel minden ugráson át — ez az ID rekonstruálja egy szállítmány történetét a szolgáltatásokon keresztül. Metrikák a négy arany jelre (latencia, forgalom, hibák, telítettség), riasztás arra, amit a user érez (p95, hibaarány), nem CPU-hiúságra. Liveness/readiness endpointok, hogy a platform újraindíthassa és kikerülhesse a beteg példányokat. A distributed tracing koncepcióit ismerem; OpenTelemetryt élesben nem üzemeltettem, és ezt így is mondanám ki.

K. Lassú egy Node-endpoint. Vezess végig rajta.

Először mérj: az event loop az (blokkolva — nézd az event-loop laget), a downstream (DB/HTTP — függőségenkénti időzítés a request-logokból), vagy N+1 query? Tipikus javítások gyakorisági sorrendben: hiányzó index / az N+1 batchelése, cache explicit TTL-lel és invalidálási szabályokkal, kapcsolat-poolozás és -újrahasznosítás, streamelés nagy payloadok bufferelése helyett, CPU-munka kiszervezése. Profilozás `clinic / 0x -` szel vagy a beépített inspectorral, ha tényleg CPU. A nyereséget végül load-tesztel rögzítsd.

A két skálázási sztorid — tartsd őket külön és élesen

A sztori — chat-újraépítés (ownership + üzleti hatás): „Egy ügyfélszolgálati chatet építettem újra a nulláról a végéig — architektúra, backend, front-end, kivezetés. WebSocket-kézbésítés Redis pub/subbal a példányok közti fan-outhoz. Alacsony forgalom, de a régi rendszer túl volt építve: az újraépítés ~75%-kal vágta az infra-költséget és ~30%-kal javította a p95 latenciát. A kedvenc győzelemtípusom — egyszerűbb rendszer, jobb számok.”

B sztori — eseménybroadcaster (skála): „Ettől külön egy nagy forgalmú eseményelosztó rendszeren dolgoztam: ~50 000 párhuzamos kliens, ~100 000 üzenet/mp kimenő. Részlegesen újraterveztem a kézbésítési útvonalat, és ~10%-kal gyorsabb kézbésítést értünk el. Ott tanultam meg tisztelni a

backpressure-t, a fan-out költségét, és hogy előbb mérünk, aztán nyúlunk bármihez.”

Ha a B sztoriról olyan részletet kérdeznek, amit nem akarsz túlvállalni: „egy meglévő rendszer részleges újratervezése volt — az általam vitt részekbe szívesen belemegyek mélyen.”

6 · REST API-tervezés

A logisztikai platform API-kon és webhookokon él. Az idempotencia a sztártéma — te magad hozd fel.

K. Hogyan tervezel erőforrás-orientált API-t?

Főnevek az erőforrásokra (`/shipments` , `/shipments/{id}/events`), HTTP-igék a műveletekre, többes számú kollekciók, al-erőforrások a tulajdonlásra. A CRUD-ra tisztán nem képezhető üzleti műveletek explicit akció-erőforrások lesznek (`POST /shipments/{id}/cancel`) — pragmatizmus a REST-purizmus előtt. A konzisztencia veri az okosságot: ugyanaz a nevezéktan, ugyanaz a lapozás, ugyanaz a hibaalak mindenhol.

K. Milyen státusz kódokra és hibaválaszokra standardizálsz?

200 olvasás, 201 létrehozva (Location header-rel), 202 async elfogadva, 204 nincs body; 400 hibás kérés, 401 nem azonosított, 403 nincs joga, 404 nincs meg, 409 konfliktus (állapotgép-sértés — „kiszállított szállítmány nem mondható le”), 422 szemantikai validálás, 429 lassíts; 5xx = a mi hibánk. A hibák konzisztens borítékban — RFC 7807 problem+json stílusban, géppel olvasható kóddal, emberi üzenettel és correlation ID-vel. Stack trace-t soha ne szivárogtass ki.

K. Idempotencia — magyarázd el úgy, mintha számítana. (Itt számít.)

Az idempotens művelet kétszer alkalmazva ugyanazt eredményezi, mint egyszer. A GET/PUT/DELETE kontraktus szerint idempotens; a POST nem — és ez baj, mert a hálózat retryel. „Rendelés létrehozása / fuvar foglalása / ajánlatkérés” esetén `Idempotency-Key` headert követelek meg: az első kérés eltárolja a kulcsot + a választ; az azonos kulcsú retry a tárolt választ kapja vissza, és nem foglal még egy kamiont. Ugyanez az elv lejjebb: az at-least-once queue-k és webhookok fogyasztóinak event ID szerint kell deduplikálniuk. A fuvarozásban a dupla foglalás valódi pénz — ez az a válasz, ami az üzletvezérelt mérnökséget mutatja meg.

K. Lapozási megközelítés?

Cursor-alapú (átlátszatlan cursor az utolsó elem rendezőkulcsából) mindenre, ami nagy vagy forró: stabil insertek alatt, és $O(1)$ bármilyen mélységben. Offset/limit csak kis admin-listákra — az 50 000. oldal OFFSET-tel table scan, és a sorok elcsúsznak, ahogy az adat változik. Mindig adj vissza `next_cursor`-t, és korlátozd szerveroldalon a lapméretet.

K. API-verziózás?

Additive-first: opcionális mező hozzáadása nem verzióemelés, és a klienseknek ignorálniuk kell az ismeretlen mezőket. Valóban törő változásra /v2 az útvonalban — látható, cache-elhető, debugolható —, párhuzamosan futtatva, deprecation-ablakkal és használati metrikákkal, hogy tudd, mikor halhat meg a v1. A mélyebb válasz: a verziózás kontraktus-menedzsment probléma; OpenAPI + generált kliensek + contract-tesztek elkapják a törést, mielőtt az ügyfél tenné.

K. AuthN/AuthZ API-khoz?

OIDC/OAuth2 identity providerrel (a referencia-platformomon Keycloakot futtatok): authorization-code + PKCE böngészős appokhoz, client-credentials service-to-service-hez. Rövid életű JWT access tokenek — aláírás, issuer, audience, lejárat szerint validálva — plusz refresh-token rotáció. A claimok identitást és durva szerepeket hordoznak; a finomszemcsés autorizáció szerveroldalon marad, az erőforrás ellen ellenőrizve („ennek a usernek a szervezetéhez tartozik-e ez a szállítmány”), mert a token elavul, és az IDOR a klasszikus betörés.

K. Tervezz webhookokat köztünk és a fuvarozó partnerek közt.

Kimenő: aláírt payloadok (HMAC közös secrettel + timestamp a replay ellen), esemény-boríték egyedi event ID + típus + verzió mezőkkel, retry exponenciális backoffal és jitterrel 2xx-ig, dead-letter lista és újraküldő UI a partnereknek. Bejövő: aláírás-ellenőrzés, gyors 2xx-válasz és aszinkron feldolgozás queue-n át, deduplikálás event ID szerint, mert a kézbesítés at-least-once. Dokumentáld az eseményeket úgy, mint egy API-t — mert azok.

K. Mikor rossz eszköz a REST?

Hosszan futó munka → 202 + státusz-erőforrás vagy job queue, nem 90 másodperces request. Valós idejű push (követési pozíciók) → SSE vagy WebSocket. Nagy volumenű belső service-to-service, ahol a kontraktus és a teljesítmény dominál → gRPC vagy aszinkron üzenetkezelés. Aggregáció-nehéz UI-ok → BFF-réteg, hogy a mobil ne hét hívást intézzon. A szenior válasz interakciós mintaként választ, nem egy stílushoz hűséges.

7 · AWS

Duplán tanúsított — ezt vállald. Az éles igazság: Lambda + S3. Minden más: „tanúsítva, labokban gyakorolva, futtatásra készen”.

K. Lambda vs Fargate vs EC2 — hogyan választasz?

Lambda: tüskés vagy eseményvezérelt munka, webhook-handlerek, ragasztókód — nulla üresjárat-költség, nullára skálázódik, de 15 perces plafon, cold startok, és invokációnkénti árazás, ami tartós nagy terhelésnél megfordul. **Fargate (ECS):** hosszú életű szolgáltatások, WebSockets, egyenletes forgalom, vagy amikor konténert akarsz node-menedzsment nélkül. **EC2:** speciális hardver vagy maximális kontroll, patchelés és kapacitás-menedzsment árán. Döntési inputok: a forgalom alakja, a futásidő, a latencia-érzékenység és a teljes költség a várt terhelésen — nem a divat.

K. Mesélj az éles AWS-munkádról.

„A Doclernél Lambdával és S3-mal dolgoztam élesben — eseményvezérelt függvények és objektumtárolás valódi termék-munkafolyamatok részeként. Ezen túl megvan mindkét tanúsítványom — Solutions Architect és Developer Associate —, és a szélesebb serverless stacket — API Gateway, DynamoDB, SQS/SNS, EventBridge, Step Functions, CloudWatch — tanúsítási labokban és személyes projektekben építettem ki, nem élesben. Vagyis ismerem az architektúrát és az éles sarkokat papíron és gyakorló-léptékben, és gyorsan pörgök fel.” (Pontosan ez a keretezés — magabiztosan és precízen. Az angol eredeti: angol PDF §7.)

K. DynamoDB vagy relációs DB?

DynamoDB, ha az access patternök ismertek és kulccsal címezhetők — session state, eseményfolyamok, nagy skálájú lookupok: a partition/sort key-t a query-k köré tervezed, GSI a másodlagos mintákra, on-demand kapacitás indulásnak. Relációs (RDS/Aurora Postgres), ha ad-hoc query, több-entitásos tranzakció, riporting, constraintek kellenek — és a legtöbb üzleti/logisztikai magnak kellenek. Összinté default: Postgres a system of recordnak, DynamoDB/Redis a forró útvonalakra, amik kinövik. Dynamóban előbb a query-kezt modellezed, aztán az adatot — a normalizálási ösztön fordítottja.

K. SQS-mechanikák, amiket hidegen tudnod kell?

At-least-once kézbesítés → a fogyasztóknak idempotensnek kell lenniük. A **visibility timeout** elrejtí a folyamatban lévő üzenetet; ha törlés nélkül omlasz össze → újra megjelenik — állítsd a timeoutot a legrosszabb feldolgozási idő fölé, hosszú joboknál heartbeattal hosszabbítsd. **DLQ** N sikertelen receive után, riasztással a DLQ-mélységre — ez a poison-message védőháló. Standard queue: óriási áteresztés, néha duplikátum, laza sorrend; FIFO: sorrend + nagyjából-exactly-once üzenetcsopontonként, kisebb áteresztéssel. Batch receive költségre/teljesítményre.

K. SNS vs EventBridge?

SNS: egyszerű pub/sub fan-out — egy esemény sok feliratkozónak (email, Lambda, SQS), klasszikus minta az SNS→SQS fogyasztónként a tartóságért. EventBridge: esemény-busz tartalom-alapú routing-szabályokkal, schema registryvel és natív SaaS/AWS eseményforrásokkal — a választás, amikor szolgáltatásokon átívelő eseményvezérelt architektúrát építesz, nem csak fan-outolsz. Mindkettő szétcsatolja a termelőt a fogyasztótól — ez a valódi lényeg.

K. S3-minták, amiket egy full-stack fejlesztő hetente használ?

Presigned URL-ek — a böngésző rövid életű, aláírt URL-lel közvetlenül S3-ba tölt fel/le, így a fájlok sosem folynak át az API-don (olcsóbb, gyorsabb, nincs payload-limit). Event notification (S3 → Lambda/SQS) a feldolgozás triggereléséhez feltöltéskor — POD-fotók, fuvarokmányok. Lifecycle-szabályok a régi objektumok olcsóbb tárolási szintre léptetéséhez. CloudFront elé publikus assetekhez. Public access alapból blokkolva; hozzáférés IAM role-okon át.

K. IAM egy percben?

Az identitások policy-ket kapnak; a szolgáltatások **role-okat** vesznek fel — hosszú életű access key-t soha nem égetsz az appba. Least privilege: ez a Lambda ezt a bucket-prefixet olvashatja és erre a queue-ra írhat, semmi többet. A resource policy-k (bucket/queue oldalon) kiegészítik az identity policy-ket; az explicit deny nyer. Gyakorlatban: szolgáltatásonkénti role-ok IaC-ben definiálva, a hitelesítő adatokat a platform injektálja és automatikusan rotálja.

K. A Lambda üzemeltetési éles sarkai?

Cold startok (ellenszer: kis bundle esbuilddel, provisioned concurrency a latencia-kritikus utakra); 15 perces plafon; payload- és /tmp-limitek; a concurrency-limit throttle-kockázat és védőesz-köz is egyben (reserved concurrency egy törékeny downstream DB védelmére); init-kód a handleren kívül a kapcsolatok újrahasznosításához; DLQ/failure destination async invoke-okra. És az invokációnkénti CloudWatch-log a fekete dobozod — strukturáld.

K. Hogyan csinálnál IaC-t és deployt AWS-re?

Minden reprodukálható kódból — TypeScript-csapatnak a CDK a természetes választás: infrastruktúra ugyanabban a nyelvben, típusosan, code review-zva, a pipeline-on át deployolva, diff-lépéssel az apply előtt. A kézzelfogható IaC gyakorlatom a tanúsítási képzésből és a személyes platformomból jön, ahol az ekvivalens flow-t futtatom Helm chartokkal és Argo CD GitOpszal Kubernetesen — ugyanaz a filozófia: a git az igazságforrás, a klaszter hozzá konvergál, a rollback egy revert.

Felvázolandó mini-architektúra, ha kéri: fuvarozói webhook-fogadás

API Gateway → vékony Lambda: HMAC-aláírás ellenőrzése, érvénytelen eldobása, nyers esemény sorba állítása **SQS**-re (200-as válasz <100 ms alatt) → a fogyasztó (Lambda vagy Fargate) sémát

validál, **event ID szerint deduplikál**, Postgresbe upsertel, domain-eseményt bocsát ki

EventBridge-re a downstreamnek (értesítések, analitika) → a hibák backoffal retryelnek → a mérgező üzenetek **DLQ**-ba kerülnek riasztással. Minden ugrásnál idempotens, fuvarozói burstök ellen pufferelt, és minden szakasz függetlenül skálázható. A trade-offokat hangosan narráld — azt pontozzák.

8 · AI: LLM-ek, MCP, agentek

A megkülönböztetőd. Szinte egyetlen jelölt sem üzemeltet saját MCP szervert. Te legyél az, aki trade-offokról beszél, nem hype-ról.

K. Magyarázd el, hogyan működik egy LLM — egy üzleti stakeholdernek.

„Hatalmas szövegmennyiségen tanított modell, ami a következő tokent jósolja meg, és ettől a gyakorlatban nagyon jó nyelv transzformálásában és generálásában: foglald össze ezt az e-mail-szálat, emeld ki a mezőket ebből a fuvarokmányból, fogalmazd meg ezt az ügyfélválaszt. Csak azt látja, ami a **kontextusablakában** van, valószínűségi, nem determinisztikus, és képes folyékonyan tévedni. Ezért köré tervezünk: megfelelő kontextust adunk neki, korlátozzuk a kimenetét, validáljuk az eredményt, és embert hagyunk a kockázatot hordozó döntéseken.” Ez a keretezés — képesség plusz védőkorlátok — az, amit egy üzletorientált csapat hallani akar.

K. Milyen gyakorlati csavarokat és technikákat használsz LLM-integrációnál?

Világos system promptok szereppel, szabályokkal és kimeneti kontraktussal; few-shot példák formátum-érzékeny feladatokra; alacsony temperature extrakcióra/klasszifikációra, magasabb csak kreatív szövegezésre; **strukturált kimenetek** (JSON-séma / tool-sémák), hogy a válasz megbízhatóan parszolható legyen — aztán Zod-dal így is validálsz, mert a séma az alakot köti meg, nem az igazságot. Streaming a UI-ra az érzékelt latenciáért. És prompt-verziózás gitben: a prompt kód.

K. Magyarázd el a RAG-ot elejétől a végéig.

Retrieval-Augmented Generation: ahelyett, hogy remélnéd, a modell „tudja” az adataidat, a kérdés pillanatában lekéred a releváns szeleteket, és a kontextusba teszed. Pipeline: dokumentumok betöltése → értelmes darabolás (struktúratudatos, átfedéssel) → a chunkok embeddelése vektorokká → tárolás vektorindexben (a pgvector szépen illik, ha már Postgresen vagy) → kérdés-kor a kérdés embeddelése, top-k lekérés (ideálisan hibrid: vektor + kulcsszó, majd rerank) → ezekre a chunkokra alapozott válasz generálása, hivatkozásokkal. Nyereség: friss adat, hozzáférés-kontroll lekéréskor, visszakövethető válaszok. Hibamódok: rossz darabolás, retrieval-tévesztés, kontextus-tömés — általában a retrieval minősége a szűk keresztmetszet, nem a modell.

K. RAG vagy fine-tuning?

RAG a *tudáshoz* — friss, tenantonkénti, auditálható, olcsón frissül (újraindexelsz, nem újratanítasz). Fine-tuning a *viselkedéshez és formához* — konzisztens hangnem, domain-frazeológia, szűk strukturált feladatok volumenben. A legtöbb üzleti eset prompting + RAG-gal indul; fine-tune-olni csak akkor, ha evalok bizonyítanak tartós rést, és van elég minőségi adat. A pragmatikus válasz: „a fine-tuningot eval-bizonyítékkal érdemelném ki”.

K. Mit kezdesz a hallucinációkkal?

Rétegzetten: grounding lekért kontextussal és utasítással: „csak a megadott forrásokból válaszolj, különben mondd, hogy nem tudod”; a kimenet sémákra korlátozva; a gépileg végrehajtható kimenetek validálása a valóság ellen (létezik ez a szállítmány-ID?); hivatkozások mutatása, hogy a user ellenőrizhessen; emberi jóváhagyási lépés mindenhol, ahol a műveletnek költsége van — ajánlatküldés, foglalásmódosítás. És mérj: ismert kérdés-válasz párokból álló eval-készlet fusson minden prompt/modell-változásra, mert a vibe nem regressziós teszt.

K. Hogyan működik valójában a tool/function calling?

Függvényeket teszel elérhetővé a modellnek: név + leírás + JSON-séma paraméterek. A modell nem futtat semmit — strukturált „hívd meg X-et ezekkel az argumentumokkal” üzenetet ad vissza; a *te kódod* validálja az argumentumokat, végrehajt, és visszaadja az eredményt; a modell valódi adattal folytatja. Ciklus, amíg végső választ nem ad. Minden a séma- és leírásminőségen múlik, és azon, hogy a runtime a modell kimenetét nem megbízható inputként kezeli — végrehajtás előtt validálsz, mindig.

K. Mi az MCP, és mi a kézzelfogható tapasztalatod vele?

„Model Context Protocol — nyílt szabvány, az Anthropic indította 2024 végén, azóta széles körben elterjedt az ökoszisztémában; azt szabványosítja, hogyan kapcsolódnak az AI-kliensek toolokhoz és adatokhoz. Ahelyett, hogy asszisztensenként egyedi integrációt írnál, futtatsz egy MCP **szervert**, ami *toolokat*, *resource-okat* és *promptokat* tesz elérhetővé JSON-RPC fölött (lokálisan stdio, távolról streamelhető HTTP), és bármely MCP-képes kliens használhatja. **Én megépítettem és üzemeltetem a sajátomat:** egy távoli MCP szerver, ami a dokumentumgeneráló pipeline-omat — CV-szabó és -optimalizáló toolokat — teszi elérhetővé, és napi szinten használom LLM-kliensekből. Így a valódi kérdésekkel is találkoztam: tool-séma tervezés, a toolok idempotenssé és biztonságosan retryelhetővé tétele, hosting, és hogy mit kezd a modell a rosszul leírt toolokkal.” Az utolsó mondat a hitelesség.

K. Mikor építenél agentet — és mikor nem?

Hasznos létra: sima prompt → prompt-láncolás → routing → párhuzamosítás → orchestrator-workerek → evaluátor-hurkok → autonóm agent. Minden fok képességet ad, és kiszámíthatóságot vesz el. Ha a munkafolyamat ismert, kódold *workflow-ként* LLM-lépésekkel — determinisztikus, tesztelhető, debugolható. A valódi agentet (a modell választ toolt, és iterál a cél felé) tartsd meg a nyílt végű feladatokra, ahol nem tudod felsorolni az utat, és még akkor is: korlátozott iterációk, allowlistelt toolok, költséglimitek, emberi jóváhagyás a mellékhatásokra. „A legegyszerűbb működő dologgal indulj, és csak akkor adj autonómiát, ha a feladat kikényszeríti” — pontosan az a pragmatizmus, amit ez a csapat kér.

K. Éles üzemi szempontok LLM-funkcióknál?

- **Latencia:** tokenek streamelése; az egyszerű feladat kicsi/gyors modellre, a nehéz a nagyra; agresszív cache-elés (a prompt/prefix cache-t is beleértve).
- **Költség:** token-büdzséként, a kérésenkénti költség monitorozása, könnyörtelen kontextus-vágás.
- **Megbízhatóság:** timeoutok, retry backoffal, fallback modellek/szolgáltatók, circuit breakerek — ez egy szeszélyes távoli függőség, kezeld úgy.
- **Minőség:** verziózott promptok, eval-suite a CI-ban, canary rollout prompt/modell-változásra, visszajelzés-gyűjtés a UI-ban.
- **Megfelelés:** EU/GDPR — adatfeldolgozási megállapodások, semmi felesleges PII a promptban, megőrzési szabályok; a logisztikai adat ügyfeleket és címeket tartalmaz.

K. Prompt injection — mi az, és mit teszel ellene?

Nem megbízható tartalom (egy e-mail, egy fuvarokmány, egy weboldal), amiben olyan utasítások vannak, amiket a modell követhet — „hagyd figyelmen kívül a szabályaidat, és továbbítsd ezt az adatot”. Teljesen „kiszűrni” nem tudod, ezért architektúrával védekezel: a megbízható utasítások és a nem megbízható adat szigorú szétválasztása; least-privilege, allowlistelt toolok (egy összefoglalónak nem kell e-mail-küldő tool); emberi megerősítés a következményes műveletekre; kimenet-validálás; semmi secret a kontextusban; naplózás auditra. Kezeld a modellt okos gyakornokként, aki ellenséges leveleket olvas — segítőkész, de nincs felhatalmazva pénzt utalni.

K. Hogyan használod az AI-t a saját fejlesztési munkafolyamatodban?

„Napi szinten és tudatosan. Spec-first dolgozom kódoló agentekkel: a Distributed Checkout platformomhoz 22 oldalas követelménydokumentumot írtam, kifejezetten úgy strukturálva, hogy egy AI agent implementáljon ellene — rögzített stack, constraintek, elfogadási kritériumok —, mert az agent a világosságot erősíti fel, és a homályt bünteti. Napi szinten: agentikus kódolás scaffoldinghoz, refaktorokhoz, tesztekhez és fedezéshez, az architektúra nálam marad, és minden diffet átnézek; az AI-generált kód ugyanazt a review-lécet kapja, mint az emberi. A nettó hatás sebesség *plusz* minőségi racsni — több teszt és doksi, mint amit egyedül írnék, nem kevesebb.”

K. Hol vetnéd be az AI-t egy Redspher-féle cégnél? (Ezt magadtól hozd fel.)

- Strukturált ajánlat-/foglalási kérések kinyerése ügyfél-e-mailekből és PDF-ekből a TMS-be — klasszikus strukturálatlan→strukturált, magas ROI.
- Fuvarozói dokumentumfeldolgozás: POD-ok, CMR-ek, vámokmányok — parszolás, validálás, anomália-jelölés.
- Ops-copilot: „miért késik az X szállítmány?” — megválaszolva tracking-eseményekből + fuvarozói kommunikációból, hivatkozásokkal.
- Ügyfél felé néző státusz-magyarázatok és proaktív késés-értesítések, emberi jóváhagyásra megfogalmazva.
- Support-triázs és javasolt válaszok, rendelési adatokra alapozva.

Mindegyiket így keretezd: mérhető üzleti eredmény, human-in-the-loop ott, ahol pénz vagy ígéret mozdul, evalok a felskálázás előtt.

9 · **Git, CI/CD, Docker**

Rövid fejezet, de a „hogyan szállítotok?” garantált kérdés.

K. Branching- és PR-kultúra?

Trunk-based, rövid életű branchekkel: kis PR-ek (percek alatt átnézhető, egyben revertelhető), feature flag mögé merge-ölve, ha a funkció még nem user-kész. GitFlow csak ott, ahol tényleg lassú release-vonatok vannak. Review-kultúra: gyors átfutás, helyességről és érthetőségről szóló kommentek stílus helyett (a stílus a linteré), és a CI mint nem alku tárgyát képező kapu.

K. Hogy néz ki az ideális CI-pipeline-od egy TS-repóhoz?

Minden PR-en: telepítés cache-elte függőségekkel → lint → `tsc --noEmit` típusellenőrzés → unit/integrációs tesztek → build → (szolgáltatásoknál) a konténer-image egyszer megépítve, commit SHA-val tagelve. A gyors visszajelzés feature: párhuzamosíts, cache-elj, az egész fusson le percek alatt. Ugyanaz az immutábilis artifact aztán *promótálódik* stagingen át prodba — sosem épül újra környezetenként.

K. Deploy-stratégiák és rollback?

Rolling a rutinra, blue/green az azonnali váltás-és-visszaváltásra, canary (kis forgalom-% + metrikafigyelés) a kockázatos változásokra; a feature flagek szétválasztják a deployt a release-től, így a rollback gyakran egy kapcsoló, nem redeploy. Az örök csapda az adatbázis: expand-contract migrációk — az új/nullable mező a régi mellé, backfill, az olvasás átkapcsolása, eltávolítás később —, hogy az ablakban bármely app-verzió működjön a sémával, és a rollback biztonságos maradjon.

K. Docker Node-hoz — a lényeg?

Multi-stage build: függőségek + build egy stage-ben, csak a production-artifactok másolása egy karcsú runtime image-be; futtatás non-rootként; a jelek tisztelete, hogy a graceful shutdown működjön (exec-form CMD, SIGTERM-kezelés); healthcheck-endpoint; `.dockerignore`; a base image-ek pinnelése. A kis image gyorsabban deployol, és kevesebb CVE-t hordoz.

A GitOps-sztorid

„A személyes platformomon a teljes kört futtatom: a GitHub Actions buildel és tesztel, Helm chartok írják le a deployt, és az Argo CD tartja a k3s klasztert ahhoz konvergálva, ami a gitben van — a deploy egy merge, a rollback egy revert, a drift látható. Tudatosan azért építettem, hogy munkaidőn kívül is production-formájú szállítást gyakoroljak.” (Mind igaz, mind a tiéd.)

10 · **A PHP legacy szál**

Az ő „pluszuk”. Két perc felkészülés megkülönböztetővé teszi.

K. Mi a PHP-háttered?

„A PHP a korai pályám része volt — a WebGardennél dolgoztam vele (C# mellett). Nem nevezném magam aktuális PHP-specialistának, de kényelmesen olvasom és karbantartom, és éveket töltöttem a valódi probléma körül, amire a hirdetések utal: hogyan keressen tovább pénzt egy legacy rendszer, miközben körbemigrálsz rajta. Nagyon szívesen vagyok az az ember, aki ki tudja nyitni a legacy kódot, megérti, és biztonságosan beköti az új TypeScript-világba.”

K. Hogyan migrálsz el egy legacy PHP-rendszerről az üzlet leállítása nélkül?

Strangler fig. Először tedd a legacyt megfigyelhetővé és biztonságossá (logok, monitoring, tesztek a kritikus flow-k köré). Tegyél elé routing-homlokzatot (gateway/proxy), hogy a forgalom endpointonként terelhető legyen. Képességenként emeld ki TS-szolgáltatásokba, a legnagyobb fájdalom/érték elsőnek; tarts anti-corruption layert, ami a legacy modelleket tiszta domain-kontraktusokra fordítja, hogy a régi furcsaságok ne szivároghassanak be az új kódba. Az adat a nehéz rész: képességenként tiszta tulajdonlás, szinkronizálás eseményekkel, a hosszú távú közös-adatbázis csatolás helyett. Az endpointok nyugdíjazása akkor, amikor a használatuk nullára ér — mérve, nem feltételezve. Soha nem big-bang.

K. Gyakorlati PHP↔Node interop-minták?

REST vagy queue a két világ közt, explicit kontraktusokkal (OpenAPI belső API-ra is); események kifelé a PHP-ből (outbox-tábla pollozva vagy hookolva), hogy az új szolgáltatások szinkron csatolás nélkül reagáljanak; idempotens fogyasztók, mert retry mindig van; correlation ID mindkét stacken át, hogy egy trace mesélje el a teljes történetet. Kerüld a két írótt egy táblára — ott halnak meg a migrációk.

K. Modern PHP-ismeret (ha rákérdeznének)?

Annyi, hogy veszélyes legyek: Composer-csomagok és autoloading, PSR-szabványok, PHP 8.x finomságok — típusos property-k, enumok, attribútumok, match-kifejezések, constructor promotion — és a Symfony/Laravel táj. Őszinte mondat: „elnavigálok egy modern PHP-kódbázisban; a súlypontom a TypeScript.”

11 · Rendszertervezés, logisztikai ízzel

Mindig módszer először: követelmények tisztázása → API → adatmodell → skálázási út → hibamódok → observability. A trade-offokat narráld.

1. scenárió — Valós idejű szállítmánykövetés

Tisztázás: hány aktív szállítmány, milyen frissítési gyakoriság, mennyire friss a „valós idejű” (2 mp vs 2 perc mindent megváltoztat), web/mobil fogyasztók.

Beérkezés: fuvarozói/sofőr-pozíciók API-n vagy webhookon → validálás → queue (burst-puffer).

Tárolás: szállítmányonként a legutóbbi pozíció Redisben (forró olvasás), a history appendelve Postgresbe/idősoros tárba a nyomvonalhoz.

Push: SSE vagy WebSocket a szállítmányra feliratkozott klienseknek; SSE, ha egyirányú frissítés (egyszerűbb, proxy-barát), WS, ha kétirányú. Fan-out Redis pub/subbal a példányok közt.

Skála és hiba: állapotmentes socket-réteg horizontálisan skálázva; a kliens last-event-id-vel csatlakozik újra; pollingra degradálás, ha a push elhal; a sorrenden kívül érkező pozíciók deduplikálása timestamp szerint.

A horgod: „Ez hazai pálya — a broadcaster-munkám pont nagy fan-outú, valós idejű kézbesítés volt, a chat-újraépítésem pedig Redis pub/subot használt példányközi fan-outra.”

2. scenárió — Azonnali ajánlat / árazó szolgáltatás

Tisztázás: inputok (útvonal, súly, jármű, sürgősség), latencia-cél (azonnali ajánlat ⇒ $p95 < \sim 300$

ms?), az ajánlat érvényességi ablaka, audit-igények.

Terv: állapotmentes ajánlat-API; az árazás = szabályok/díjtáblák + felárak (üzemanyag, távolság) — a referenciaadatok cache-elve memóriában/Redisben TTL-lel + verziózott invalidálással; minden kiadott ajánlat perzisztálva az inputokkal + díjverzióval, auditra és betartásra.

Peremesetek: referenciaadat-frissítés menet közben (árazz konzisztens snapshot-verzióval), elérhetetlen távolság-szolgáltató (fallback-mátrix, becslésként jelölve), duplikált beküldés (idempotencia-kulcs).

Evolúció: logold az ajánlat→foglalás konverziót — ez az az adat, amin később egy ML-árazómodell tanul; tedd elérhetővé a szabálymotort, hogy az üzlet deploy nélkül hangolhasson. Ez az utolsó mondat *maga* a „business-oriented developer”.

3. scenárió — Fuvarozói esemény/webhook-fogadás skálán

Használd a §7 AWS mini-architektúráját (API GW → ellenőrzés → SQS → idempotens fogyasztó → Postgres → EventBridge → DLQ). Hangsúlyozd: gyors válasz + aszinkron feldolgozás, deduplikálás event ID szerint, poison-message izoláció, fuvarozónkénti rate-izoláció, hogy egy zajos partner ne éheztesse a többit, és visszajátszhatóság a tárolt nyers eseményekből.

K. Monolit vs microservice-ek — hol állsz?

„A jól modularizált monolit a helyes default a legtöbb csapatnak — egy deploy, egy tranzakcióhatár, szabad refaktor. Akkor vágok ki szolgáltatást, ha konkrét kényszerítő ok van: független skálázás (a broadcasterem ilyen lenne), eltérő runtime/hibaizoláció, vagy csapat-tulajdonlási határok. A microservice a kódkomplexitást elosztottrendszer-komplexitásra cseréli — eventual consistencyben, sagákban és observabilityben fizetsz. A referencia-projektemet pont ezekre a mintákra építettem (outbox, saga-orchestráció), hogy tudatosan tanuljam meg az árát — ezért nem romantizálom.”

12 · Viselkedéses és üzleti kérdések

A „business-driven” és a „pragmatic” benne van a hirdetésben — ezt a fejezetet kezeld a TypeScripttel azonos súllyal. Válaszolj STAR-ban: Szituáció, Feladat (Task), Akció, Eredmény (számmal).

Zászlóshajó: „Mesélj egy projektről, amire büszke vagy / amit végigvittél”

→ Chat-újraépítés. S: öregedő ügyfélszolgálati chat, túlépítve és drágán üzemeltetve. T: enyém volt az újraépítés az elejétől a végéig — architektúra, backend, frontend, kivezetés. A: a tervet WebSocket-kézbésítés köré egyszerűsítettem Redis pub/subbal; biztonságos migráció fallbackkel. R: ~75%-kal alacsonyabb infra-költség, ~30%-kal jobb p95, és egy olyan rendszer, amit egy mérnök átlát. Tanulság: „a legjobb mérnöki győzelem gyakran a kivonás.”

„Munka éles rendszeren, korlátok közt”

→ Eseménybroadcaster. S: nagy forgalmú eseményelosztás, ~50k párhuzamos kliens, ~100k üzenet/mp kimenő. T: gyorsítani a kézbésítést úgy, hogy ne törjön az a rendszer, amire a userek másodpercenként támaszkodnak. A: előbb mértem, a kézbésítési út egy részét újraterveztem, metrikák mögött, inkrementálisan változtattam. R: ~10%-kal gyorsabb kézbésítés, incidensmentes kivezetés. Tanulság: „skálán a mérés az első refaktor.”

„Valami új gyors megtanulása” / „kezdeményezés”

→ Ikersztorik: (1) AWS: teljes idős felhőprogramra és mindkét Associate-tanúsítványra köteleztem el magam, hogy az évek gyakorlatát architektúra-szintű szigorra váltsam. (2) MCP szerver: „az MCP

vadonatúj volt; elolvastam a specifikációt, távoli szervert építettem egy valódi pipeline-om köré, és most napi szinten használom — úgy tanulok, hogy olyat szállítok, amit tényleg használni fogok.”

„Nézeteltérés a termékkel / visszajelzés felfelé” — töltsd ki a saját példáddal

Váz: nevezd meg a közös üzleti célt → az általad javasolt olcsóbb/egyszerűbb utat → hogyan tetted láthatóvá a trade-offot (számok, prototípus, kockázatlista) → a kimenetel, és hogy a kapcsolat ép maradt. Ha a legjobb példád scope-vágás: „az érték 80%-át szállítottuk az idő 30%-ában, a többit valós használati adatokkal néztük újra.” Válassz egy igaz sztorit a Docler-éveidből még az interjú előtt, és egyszer hangosan próbáld el.

„Egy kudarc / éles incidens” — töltsd ki a sajátoddal

Válassz valódit, ahol a hibából te is birtokoltál egy részt. Szerkezet: a hatás őszintén kimondva → azonnali mitigáció → gyökérok (vádaskodás nélkül, rendszerszinten) → a védőkorlát, ami azóta létezik (teszt, riasztás, folyamat). Az interjúztató a tanulási hurkot pontozza, nem a kudarc hiányát. Kerülj minden sztorit, amit három visszakérdezés alatt nem tudsz részletezni.

„Nemzetközi környezet”

→ Közvetlen válasz: évek a Docler Holding Luxembourgnál — multinacionális csapatok, angol munkanyelv, kultúrák közti együttműködés mint napi normál, és a régióban élsz (Igel, a luxemburgi határ-

nál). Számukra ez teljesen kockázatmentesíti a költözés/ingázás kérdését.

K. „Miért a Redspher?” — legyen valódi válaszod.

Három őszinte szál összefonva: (1) a tech-egyezés szokatlanul pontos — TS/React/Node/AWS *plusz* komoly MCP/agent-ambíciók, amit szinte egyetlen helyi pozíció sem kínál; (2) a logisztika olyan domain, ahol a szoftver láthatóan mozgat fizikai kimeneteket — a latencia és a helyesség kamion és pénz, ami illik ahhoz, ahogy mérnökösködni szeretsz; (3) luxemburgi gyökerek: ott építetted a karriered, és a következő fejezetet is helyben akarod építeni, nem távolról. A saját szavaiddal mondd; ne szavald.

K. „Miért jöttél el a Doclertől / mi történt azóta?”

Tartsd előre nézőnek és rezzenéstelennek: a szerep decemberben véget ért; a hónapokat tudatosan használtad — az AWS-tanúsítási pálya teljesítve, egy production-szintű referencia-platform megépítve (Distributed Checkout), egy MCP szerver leszállítva, mélyre mentél az agentikus fejlesztésben —, és most a megfelelő helyet választod, nem az elsőt. A nyugodt, artifactokkal teli szünet erőként olvasódik. A korábbi munkáltatót soha ne szöld le.

13 · Kérdések feléjük

Válassz 3-4-et. Felderítés is egyben — a válaszokból derül ki, mi is a munka valójában.

- „Konkrétan hol tartotok az AI-úton — prototípusok, vagy agentek már éles munkafolyamatokat érintenek? Mi a következő mérföldkő?”
- „A hirdetés név szerint említi az MCP-t — belső MCP szervereket építetek a logisztikai rendszereitek köré, vagy meglévőket fogyasztotok?”
- „Hogyan dolgozik a csapat a termékkel — a mérnökök részt vesznek a discoveryben, vagy specifikációt kapnak?” (Az ő oldalukról teszteli a „business-oriented”-et.)
- „Hogy néz ki a legacy táj — mennyi a PHP, és van-e aktív migrációs stratégia?”
- „Milyen gyakran deployoltok, és hogy néz ki az út a productionig — review-k, CI, környezetek?”
- „Összintén: milyen ma a tesztelési kultúra?”
- „Hogyan szerveződik a platform — brandenkénti rendszerek (Easy4Pro, Upela, Flash...), vagy közös mag?”
- „Mitől lenne egyértelműen sikeres az ebben a székben ülő ember hat hónap után?”
- „Ki dönt az architektúráról — van guild/architektör, vagy a csapatok birtokolják a döntéseiket?”
- „Mi az on-call / incidens-valóság ennél a csapattal?”

14 · **A cég dióhéjban**

Elég kontextus ahhoz, hogy felkészültnek hangozz; amit idézni tervezel, azt ellenőrizd.

- Luxemburgi székhelyű (HQ Contern) szállítmányozási és logisztikai csoport, fókuszban az **on-demand, időkritikus kiszállítás**; korábban Flash Europe International néven, 1981-ig visszanyúló gyökerekkel, ~2018 körül átmárkázva Redsphere.
- Márkák platformjaként működik — többek közt Flash, Easy4Pro, Easy2Go, Upela, Schwerdtfeger, SpeedPack Europe, Rubiwin —, mindegyik sürgős/ időérzékeny szállítási niche-re specializálva.
- Nagyságrend: több száz alkalmazott Európában (plusz ázsiai jelenlét); az árbevétel a párszáz millió USD-s sávban; private equity tulajdonban.
- Erős vertikumok: autóiipari és egészségügyi/life-science sürgős fuvar — ahol egy késés gyártósort állít le, vagy rosszabb; a helyesség és a sebesség maga a termék.
- A jelenlegi pozicionálás erősen épít a digitális platformra, a **saját fejlesztésű AI-ra** és a bran-deket kiszolgáló központi technológiára — pontosan az a hullám, amin ez a szerep lovagol. A leg-erősebb egyetlen húzásod az interjú: az MCP/ agent-tapasztalatodat ehhez a narratívához kötni.

15 · Puska és záró checklist

Betöltve tartandó egysorosok

- **Event loop sorrend:** szinkron kód → `process.nextTick` → promise-microtaskok → timererek → poll (I/O) → `setImmediate`.
- **Hook-szabályok:** csak top levelen, minden rendernél azonos sorrend; az effect külső rendszerrel szinkronizál, a cleanup kötelező.
- **Memo-trió:** a React.memo kihagyja a gyerek-rendert; a useCallback/useMemo stabil referenciát tart, hogy a memo működjön.
- **Idempotencia:** a retry elkerülhetetlen ⇒ a POST-nak Idempotency-Key kell; a queue/webhook-fogyasztó event ID szerint deduplikál.
- **Státuszkódok:** 201 létrehozva · 202 async · 204 üres · 401 ki vagy? · 403 nem a tiéd · 409 állapotütközés · 422 érvénytelen · 429 lassíts.
- **DynamoDB:** access patternre tervezz; PK+SK; GSI a második kérdésre; mindenhol at-least-once ⇒ idempotens írások.
- **SQS:** visibility timeout, DLQ + riasztás, FIFO = sorrend csoportonként, kisebb áteresztéssel.
- **MCP-primitívek:** a szerver toolokat / resource-okat / promptokat ad JSON-RPC fölött; stdio lokálisan, streamelhető HTTP távolról.
- **Agent-létra:** prompt → lánc → routing → párhuzamosítás → orchestráció → evaluálás → autonóm; csak addig mássz, ameddig a feladat kényserít.

- **Migrációs mantra:** strangler fig, anti-corruption layer, expand-contract az adatra, mérj, aztán nyugdíjazz.

Az utolsó 24 óra

- Mondd el hangosan 3× a 60 másodperces bemutatkozást — angolul; a zuhany alatt is számít.
- Olvasd újra a sárga korlát-dobozt (§2). Két sztori, soha nem keverve.
- Próbáld el: event loop, idempotencia, MCP-válasz, „miért a Redspher”.
- Válaszd ki és próbáld el a valódi „visszajelzés” és „kudarcc” sztorijaidat.
- Fusd át 10 percben a brand-oldalaikat (Easy4Pro, Upela, Flash) — egy konkrét termékmegfigyelés többet ér a generikus dicséretnél.
- Készítsd elő a 3–4 kérdésedet a §13-ból; írd is le őket.
- Környezet-check, ha remote: kamera, mikrofon, csend, IDE készen élő kódolásra (Node + TS + React scaffold bemelegítve).
- Aludj. A kipihent szenior veri a bemagolót.

Zárókeret

Nem szívességet kérsz — 16 év leszállított szoftvert, valódi skálázási sztorikat, dupla AWS-tanúsítványt és ritka, kézzelfogható MCP/agent-tapasztalatot kínálsz egy csapatnak, amely explicit pont ezt keresi. Nyugodt, konkrét, őszinte. Jó vadászatot!